

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Deep Learning - MCSE603L

Reg No. : 23MAI0014

Name : Harini K

Slot : F1

Faculty : Dr.Anil Kumar K

College : Vellore Institute of Technology

Department : SCOPE

Assignment : 1

Experiment : (i) Autoencoders

(ii) DALL-E

(iii) DALL-E2

(iv) IMAGEN

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

DIGITAL ASSIGNMENT – 1

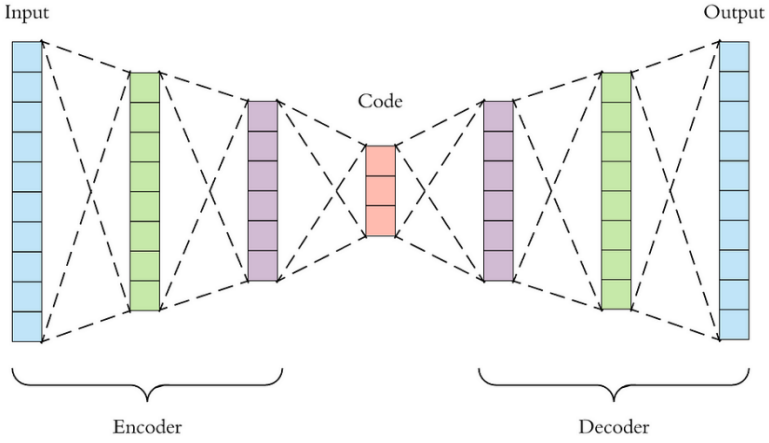
(I) AUTOENCODERS

Autoencoders are a specific type of feedforward neural networks where the input is the same as the output. They compress the input into a lower-dimensional code and then reconstruct the output from this representation. The code is a compact "summary" or "compression" of the input, also called the latent-space representation.

An autoencoder consists of 3 components: encoder, code and decoder. The encoder compresses the input and produces the code, the decoder then reconstructs the input only using this code.

Architecture

Both the encoder and decoder are fully-connected feedforward neural networks, essentially the ANNs. Code is a single layer of an ANN with the dimensionality of our choice. The number of nodes in the code layer (code size) is a hyperparameter that we set before training the autoencoder.



First the input passes through the encoder, which is a fully-connected ANN, to produce the code. The decoder, which has the similar ANN structure, then produces the output only using the code. The goal is to get an output identical with the input. Note that the decoder architecture is the mirror image of the encoder. This is not a requirement but it's typically the case. The only requirement is the dimensionality of the input and output needs to be the same. Anything in the middle can be played with.

There are 4 hyperparameters that we need to set before training an autoencoder:

Code size: number of nodes in the middle layer. Smaller size results in more compression.

Number of layers: the autoencoder can be as deep as we like. In the figure above we have 2 layers in both the encoder and decoder, without considering the input and output.

Number of nodes per layer: the autoencoder architecture we're working on is called a stacked autoencoder since the layers are stacked one after another. Usually stacked autoencoders look like a "sandwitch". The number of nodes per layer decreases with each subsequent layer of the encoder, and increases back in the decoder. Also the decoder is symmetric to the encoder in terms of layer structure. As noted above this is not necessary and we have total control over these parameters.

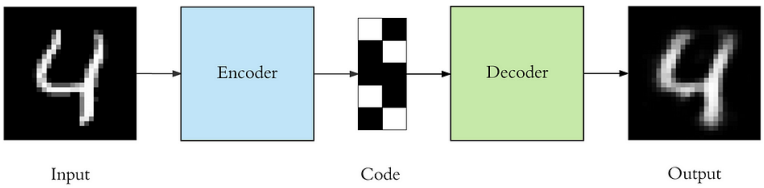
Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Loss function: we either use mean squared error (mse) or binary crossentropy. If the input values are in the range [0, 1] then we typically use crossentropy, otherwise we use the mean squared error. For more details check out this video.

Autoencoders are trained the same way as ANNs via backpropagation.



Uses of autoencoders

1. Data Compression - By acquiring a compact representation of the input data, autoencoders can be applied to data compression tasks.
2. Dimensionality Reduction - They are helpful for preprocessing or visualization since they can reduce the dimensionality of data while maintaining its key characteristics.
3. Anomaly Detection - By recreating the input data and labeling variations from the learned patterns as anomalies, autoencoders can identify anomalies in data.
4. Generative Models - One class of autoencoder that can produce fresh data samples that resemble training data is the variational autoencoder (VAE), which can learn a probabilistic distribution of the data.

Strengths and weaknesses

Strengths: Effective for dimensionality reduction, feature learning, and data denoising. Can learn meaningful representations without labeled data.

Weaknesses: Limited by the capacity of the latent space, prone to overfitting, sensitive to hyperparameters.

Algorithm and flow chart

Data Preparation

The first step is to prepare your dataset. Make sure it has been correctly normalized and preprocessed.

Describe architecture
Choose your autoencoder's architecture, taking into account the latent space dimension, activation functions, and number of layers. An encoder and a decoder are the two main components of autoencoders. While the decoder reconstructs the original input from the encoded representation, the encoder lowers the dimensionality of the input data.

Construct the Model
Use the deep learning framework of your choice (e.g., TensorFlow, PyTorch) to implement the encoder and decoder.
Combine the encoder and decoder to create the autoencoder.

Put the Model Together
Give the loss function a definition. Depending on the type of input data, mean squared error (MSE) or binary cross-entropy are two popular loss functions for autoencoders.
Select the optimizer (such as Adam or SGD) and establish the learning rate.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Educate the Model
Provide the autoencoder with the supplied data.
Reduce the reconstruction error by using backpropagation to update the model's parameters.
Continue iterating throughout several epochs until convergence.
To prevent overfitting, keep an eye on the reconstruction loss on a validation set.

Assessment
Assess the trained autoencoder's performance on an independent test set.
Evaluate the quality of the reconstruction and any additional pertinent metrics.

Applications
Apply the learned autoencoder to several tasks like data generation, dimensionality reduction, anomaly detection, and denoising.

Fine-tuning
If necessary, adjust the autoencoder's performance or investigate alternative architectures.

Code:

```
%matplotlib inline
import matplotlib
import matplotlib.pyplot as plt

from keras.layers import Input, Dense
from keras.models import Model

# this is the size of our encoded representations
encoding_dim = 32  # 32 floats -> compression of factor 24.5, assuming
the input is 784 floats

# this is our input placeholder
input_img = Input(shape=(784,))
# "encoded" is the encoded representation of the input
encoded = Dense(encoding_dim, activation='relu')(input_img)
# "decoded" is the lossy reconstruction of the input
decoded = Dense(784, activation='sigmoid')(encoded)

# this model maps an input to its reconstruction
autoencoder = Model(inputs = input_img, outputs=decoded)
print("autoencoder model created")
# this model maps an input to its encoded representation
encoder = Model(inputs=input_img, outputs=encoded)

# create a placeholder for an encoded (32-dimensional) input
encoded_input = Input(shape=(encoding_dim,))
# retrieve the last layer of the autoencoder model
decoder_layer = autoencoder.layers[-1]
# create the decoder model
```

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

```
decoder = Model(inputs=encoded_input,
outputs=decoder_layer(encoded_input))
autoencoder.compile(optimizer='adam', loss='binary_crossentropy',
metrics = ['accuracy'])

from keras.datasets import mnist
import numpy as np
(x_train, _), (x_test, _) = mnist.load_data()

# plt.imshow(x_train[2001], cmap='gray')

x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(x_train.shape[1:])))
x_test = x_test.reshape((len(x_test), np.prod(x_test.shape[1:])))
print(x_train.shape)
print(x_test.shape)


autoencoder.fit(x_train, x_train,
                epochs=50,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))

# encode and decode some digits
# note that we take them from the *test* set
encoded_imgs = encoder.predict(x_test)
decoded_imgs = decoder.predict(encoded_imgs)

n = 10  # how many digits we will display
plt.figure(figsize=(20, 4))
for i in range(10):
    # display original
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)

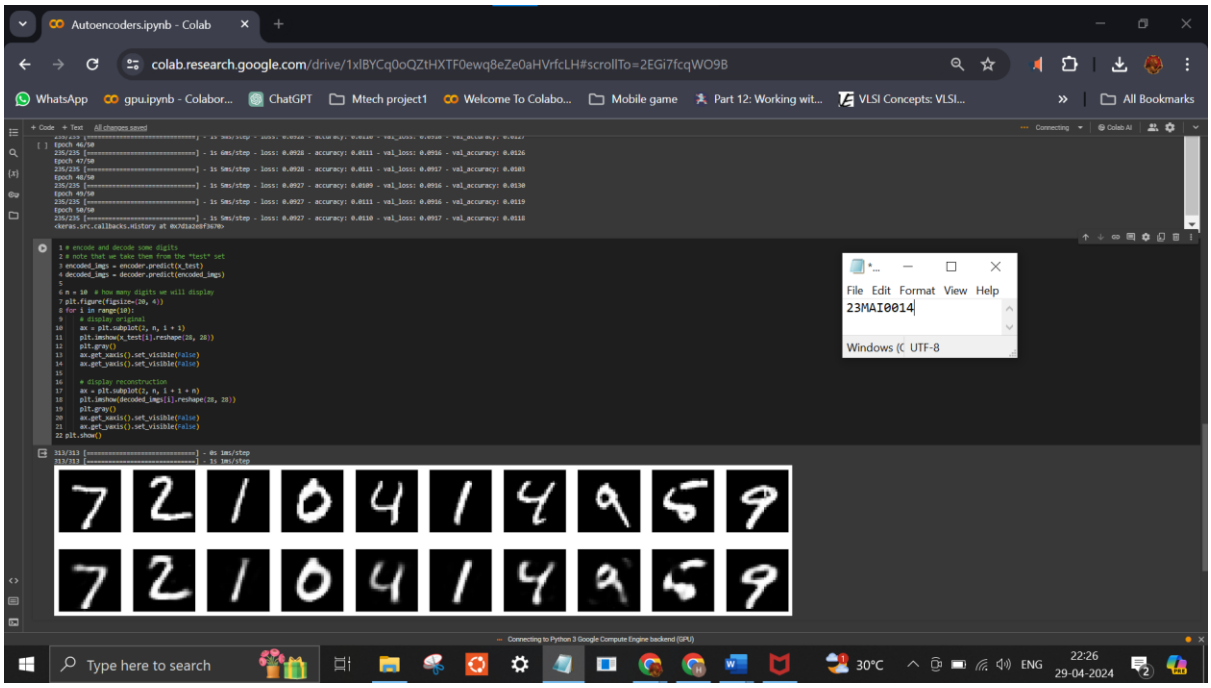
    # display reconstruction
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28))
    plt.gray()
    ax.get_xaxis().set_visible(False)
    ax.get_yaxis().set_visible(False)
plt.show()
```

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Execution results



Dataset

Mnist dataset - It consists of a large set of grayscale images of handwritten digits from 0 to 9. Each image is a 28x28 pixel grid, making it a relatively small and manageable dataset. The dataset is widely used for training and testing various machine learning algorithms, particularly those related to image classification. The MNIST dataset typically contains 60,000 training images and 10,000 test images. Each image is accompanied by its corresponding label, indicating the digit it represents (0 to 9).

Google Colab Link

https://colab.research.google.com/drive/1xlBYCq0oQZtHXTF0ewq8eZe0aHVrfcLH?usp=sharing

Reference

https://towardsdatascience.com/applied-deep-learning-part-3-autoencoders-1c083af4d798

https://www.geeksforgeeks.org/auto-encoders/

Conclusion

Autoencoders stand as versatile tools in the realm of machine learning, offering a robust framework for unsupervised learning, data compression, and feature learning. Their ability to capture meaningful representations of input data has fueled advancements across various domains, from anomaly detection to transfer learning. Despite their successes, challenges such as overfitting and training instability persist, underscoring the need for ongoing research and innovation in model development. As researchers continue to refine autoencoder architectures and training techniques, these models are poised to play an increasingly pivotal role in driving advancements in artificial intelligence and data analysis.

Deep Learning - MCSE603L

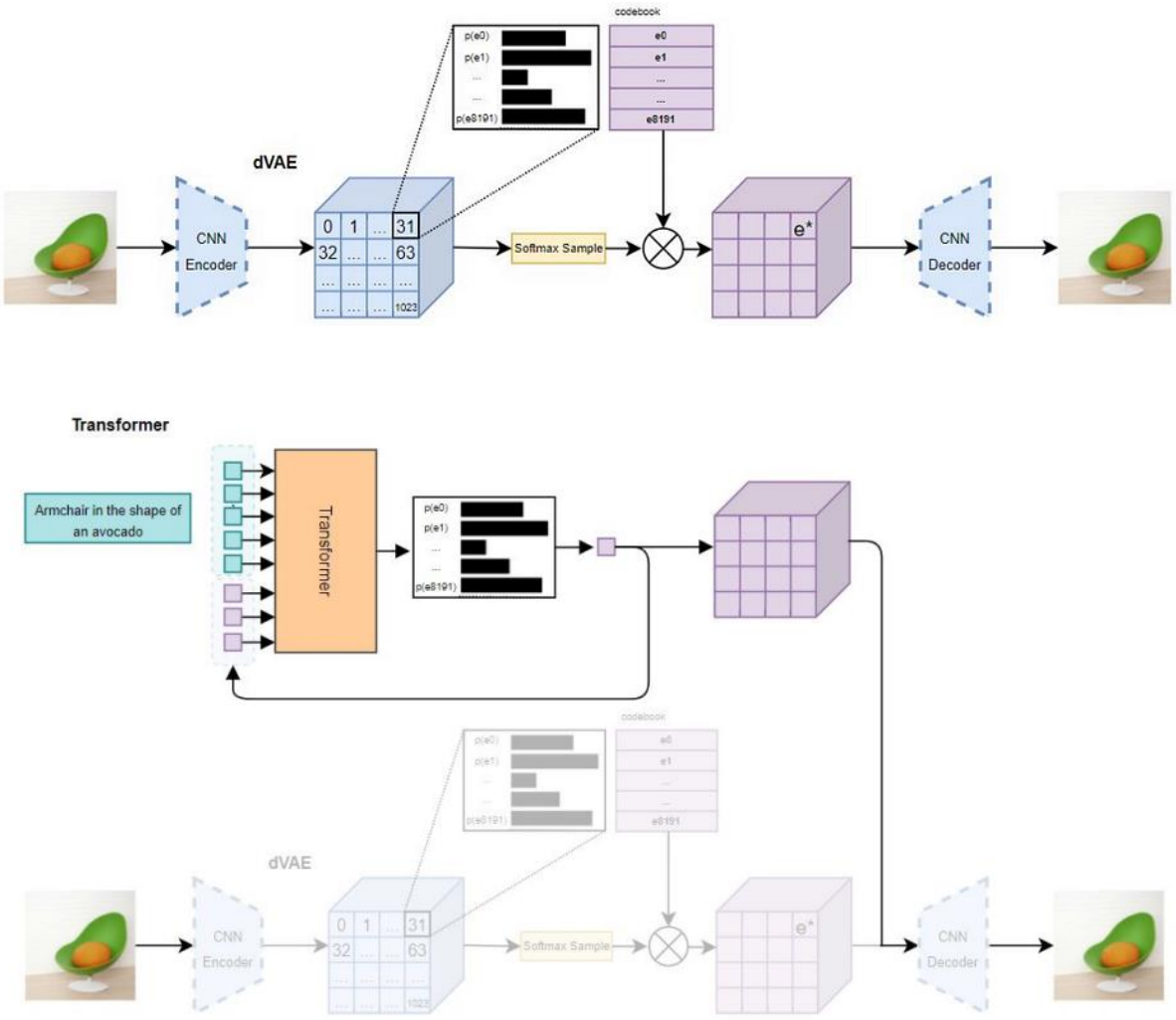
Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

(II) DALL-E

Dall-E, developed by OpenAI, is an innovative generative AI technology designed to create images based on text prompts. Released in January 2021, Dall-E leverages neural networks, natural language processing (NLP), and large language models (LLMs) to generate entirely new images in various styles specified by the user's prompts. The name "Dall-E" pays homage to the fusion of art and AI, with "DALL" evoking Salvador Dali's surrealism and "E" referencing the Disney robot Wall-E, symbolizing the automated creativity of the technology.

Architecture



Dall-E employs a subset of the GPT-3 large language model (LLM), utilizing 12 billion parameters optimized for image generation. It utilizes transformer neural networks to comprehend and create connections between different concepts from natural language prompts. The core approach enabling Dall-E, termed Zero-Shot Text-to-Image Generation, was detailed in a research paper by OpenAI researchers. The model's architecture also involves a Discrete Variational Auto-Encoder (dVAE) for the initial Dall-E iteration and a diffusion model in Dall-E 2, which generates higher-quality and photorealistic images.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Dall-E interprets user prompts through NLP, understanding concepts and contexts to generate corresponding images. The model utilizes prior knowledge and related concepts to execute tasks, such as creating entirely new images, based on the input text. CLIP (Contrastive Language-Image Pre-training) aids in evaluating Dall-E's output by analyzing suitable captions for generated images. Through iterations and refinements, Dall-E continuously improves its image generation capabilities.

Uses of Dall-E

Dall-E offers a wide range of applications, including creative inspiration, entertainment (e.g., books, games), education (e.g., visual aids for concepts), advertising, marketing, product design visualization, art creation, and fashion design. It enables users to quickly and easily generate customized images based on text prompts, fostering creativity and innovation across various domains.

Strengths and Weaknesses of Dall-E

Strengths:

Speed: Dall-E rapidly produces images from simple text prompts in less than a minute.

Customization: Users can create highly customized images tailored to their imagination through text prompts.

Accessibility: Dall-E is accessible to users, requiring only natural language text input without extensive training or programming skills.

Extensibility: The technology facilitates the extension and re-imagination of existing images, fostering iterative creativity.

Weaknesses:

Copyright Concerns: There are issues regarding copyright on Dall-E-generated images and whether the model was trained on copyrighted material.

Legitimacy of AI-generated Art: Some question the legitimacy and ethical implications of AI-generated art, potentially displacing human creators.

Data Set Limitations: Despite training on a large dataset, Dall-E may not generate intended images if the prompt lacks foundational information or context.

Realism: While Dall-E 2 enhances image quality, some generated images may still lack realism for certain users, limiting their utility in specific contexts.

Algorithm and flow chart

Input Text Prompt:

Receive a text prompt describing the desired image from the user.

Tokenization:

Tokenize the input text prompt, breaking it down into discrete tokens representing words, concepts, and attributes.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Preprocessing:

Preprocess the tokens representing the text prompt, encoding them into a fixed-size vector representation suitable for input into the neural network.

Neural Network Architecture:

Utilize a transformer-based neural network architecture similar to GPT-3, designed to process both text and image inputs as a single stream of data.

Data Input:

Combine the vector representation of the text prompt with the vector representation of an empty image as input to the neural network.

Model Training:

Train the neural network using maximum likelihood estimation to generate tokens sequentially, one after another, to construct the image based on the input text prompt.

Image Generation:

Generate the image token by token, leveraging the transformer's autoregressive nature to predict the next token in the sequence based on the preceding tokens.

Postprocessing:

Decode the generated image tokens into a visual representation, reconstructing the image from the discrete latent codes generated by the neural network.

Output Image:

Output the generated image corresponding to the input text prompt, presenting it to the user for review and further refinement if necessary.

Evaluation and Refinement:

Evaluate the quality and relevance of the generated image compared to the input text prompt.

Refine the model's training data and parameters based on user feedback to improve future image generation.

Repeat Process:

Repeat the process iteratively, allowing users to generate multiple images from different text prompts and refine the model continuously.

This algorithm outlines the fundamental steps involved in how DALL·E processes text prompts and generates corresponding images, leveraging transformer-based neural network architecture and autoregressive generation techniques.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Name: Harini K
Slot: F1
Reg no. 23MAI0014
Technical Part

1. Transformer Design:

DALL·E adopts a transformer architecture, akin to models like GPT (Generative Pre-trained Transformer), comprising feedforward neural networks and layers of self-attention processes. This design facilitates the effective capture of long-range dependencies in both textual and visual data.

2. Text Encoder:

Equipped with a text encoder, DALL·E accepts textual descriptions as input. The text encoder transforms the input text into a high-dimensional vector representation, preserving contextual information and semantic meaning. Subsequent layers further process this representation.

3. Image Decoder:

DALL·E incorporates an image decoder responsible for generating images based on the encoded textual descriptions. Leveraging text embeddings created by the text encoder, the image decoder translates them into pixel-level image representations. Multiple neural network layers are employed in this decoding process to convert text embeddings into cohesive image outputs.

4. Multi-Modal Fusion:

Employing multi-modal fusion techniques, DALL·E seamlessly combines textual and visual information. Through this fusion process, visual features derived from the image decoder are integrated with semantic information from textual descriptions. This integration ensures visually cohesive and consistent output aligned with the provided textual descriptions.

5. Training Process:

DALL·E undergoes training using a vast collection of matched text descriptions and related images sourced from the internet. During training, the model minimizes a loss function measuring the disparity between generated and ground truth images. Optimization techniques like gradient descent and backpropagation adjust the model's parameters to produce images corresponding to provided textual descriptions.

6. Adjusting and Transferring Knowledge:

DALL·E's image generation capabilities can be fine-tuned to specific tasks or domains by adjusting the model on particular datasets. Fine-tuning involves updating the model's parameters using additional data from the target domain, enabling it to capture domain-specific characteristics and enhance performance on specific tasks.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Code

```
! pip install min-dalle
! nvidia-smi
dtype = "float32" #@param ["float32", "float16", "bfloat16"]
from IPython.display import display, update_display
import torch
from min_dalle import MinDalle

model = MinDalle(
    dtype=getattr(torch, dtype),
    device='cuda',
    is_mega=True,
    is_reusable=True
)
%%time

text = "Milky way galaxy" #@param {type:"string"}
progressive_outputs = True #@param {type:"boolean"}
seamless = False #@param {type:"boolean"}
grid_size = 3 #@param {type:"integer"}
temperature = 1.00 #@param {type:"slider", min:0.01, max:16, step:0.01}
supercondition_factor = 16 #@param {type:"number"}
top_k = 128 #@param {type:"integer"}

image_stream = model.generate_image_stream(
    text=text,
    seed=-1,
    grid_size=grid_size,
    progressive_outputs=progressive_outputs,
    is_seamless=seamless,
    temperature=temperature,
    top_k=int(top_k),
    supercondition_factor=float(supercondition_factor)
)

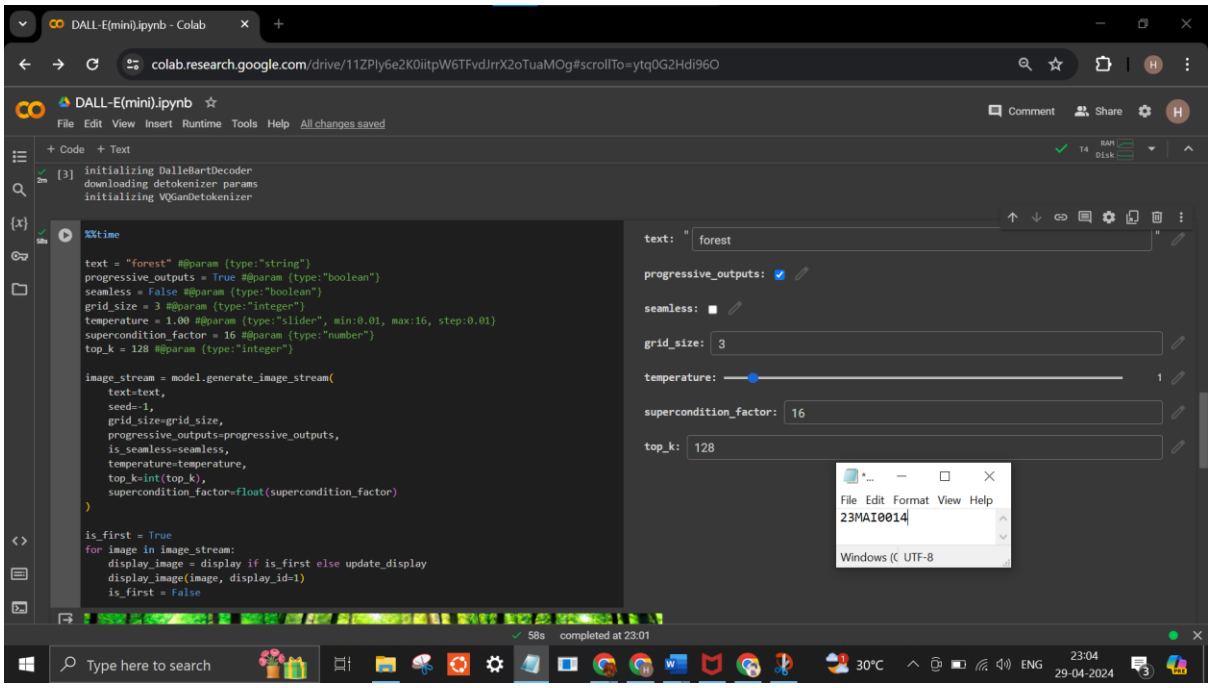
is_first = True
for image in image_stream:
    display_image = display if is_first else update_display
    display_image(image, display_id=1)
    is_first = False
```

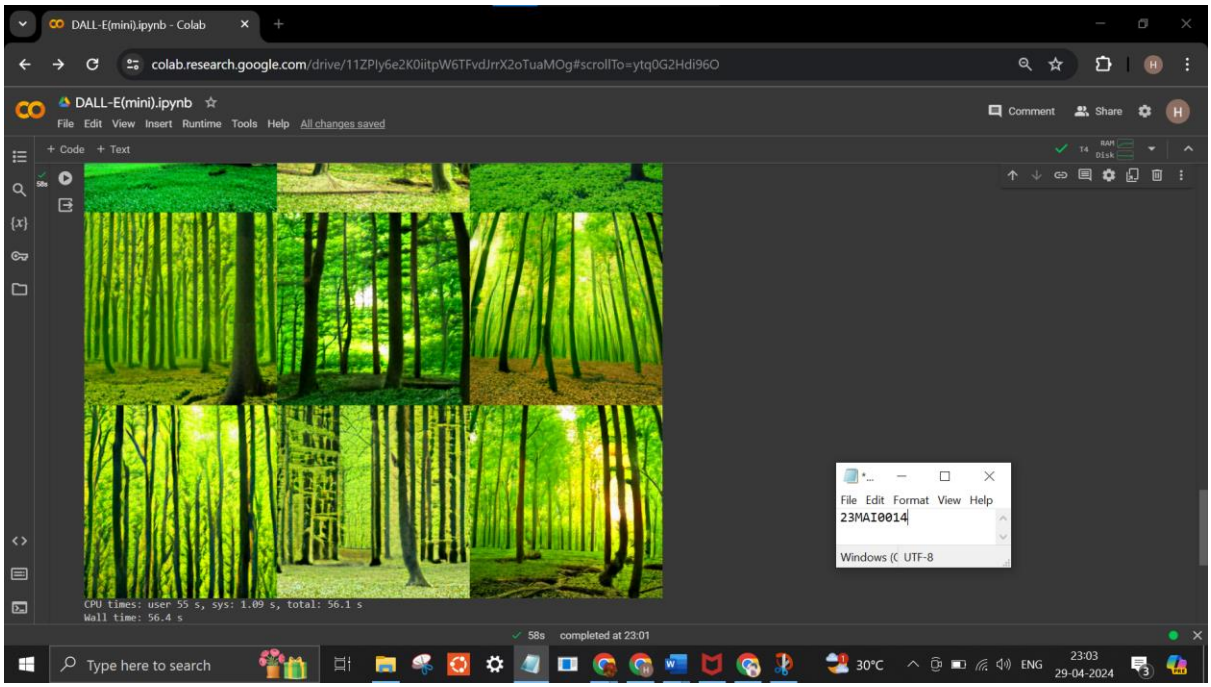
Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Execution results





Google Colab Link

https://colab.research.google.com/drive/11ZPIy6e2K0iitpW6TFvdJrrX2oTuaMOg?usp=sharing

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Reference

https://openai.com/research/dall-e

https://medium.com/@eeman.majumder/dall-e-api-just-got-launched-here-is-how-to-use-it-afdffc2f3a9

https://medium.com/@zaiinn440/how-openais-dall-e-works-da24ac6c12fa

https://github.com/simonsanvil/DALL-E-Explained/blob/main/DALL_E_CLIP_VAE_Explained.ipynb

https://colab.research.google.com/github/cedro3/others/blob/master/DALL_e_sample.ipynb

https://www.techtarget.com/searchenterpriseai/definition/Dall-E

Conclusion

DALL·E represents a significant advancement in text-to-image synthesis, building upon previous research in this area. It adopts a decoder-only transformer architecture and employs attention mechanisms to model the relationship between text and image tokens. The model's training procedure is influenced by prior work on GANs, contrastive learning, attention mechanisms, and multimodal discriminative models. DALL·E's integration with CLIP for reranking samples further enhances sample quality and demonstrates the potential of language-guided search in generative AI.

In conclusion, DALL·E stands at the forefront of generative AI technology, showcasing the remarkable fusion of natural language processing and image generation. Its ability to translate textual descriptions into intricate and diverse images opens up a world of possibilities across various domains, from creative expression to practical applications in design, education, entertainment, and beyond. By enabling users to effortlessly generate custom images from simple text prompts, DALL·E streamlines creative workflows and empowers individuals and organizations to explore, innovate, and communicate visually in unprecedented ways.

However, as with any transformative technology, DALL·E also raises important ethical considerations, including copyright concerns, potential biases in model outputs, and the impact on human creativity and labor. Moving forward, it will be crucial for researchers, developers, and policymakers to address these challenges and ensure that DALL·E's potential benefits are maximized while mitigating potential risks. Nonetheless, DALL·E represents a significant milestone in AI research and holds promise for revolutionizing how we interact with and create visual content in the digital age.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

(III) DALL-E2

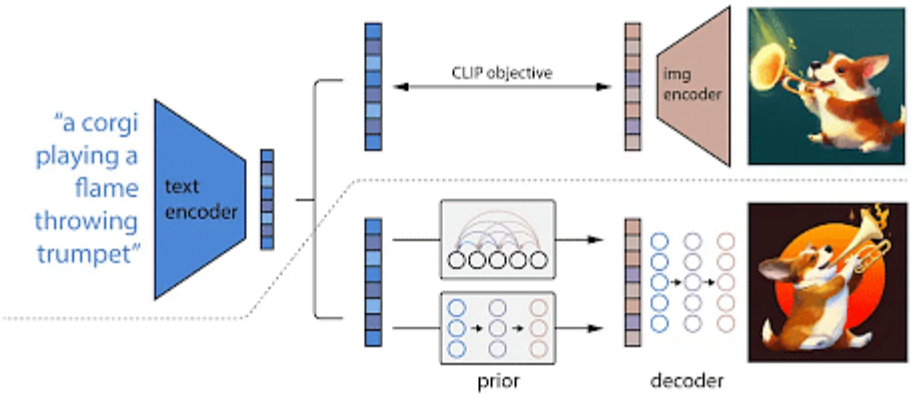
DALL-E 2 is a state-of-the-art AI-powered image-generating platform created by OpenAI. It is the advanced and improved successor of DALL-E. It can produce incredibly realistic and detailed pictures based on textual descriptions by utilizing deep learning techniques.

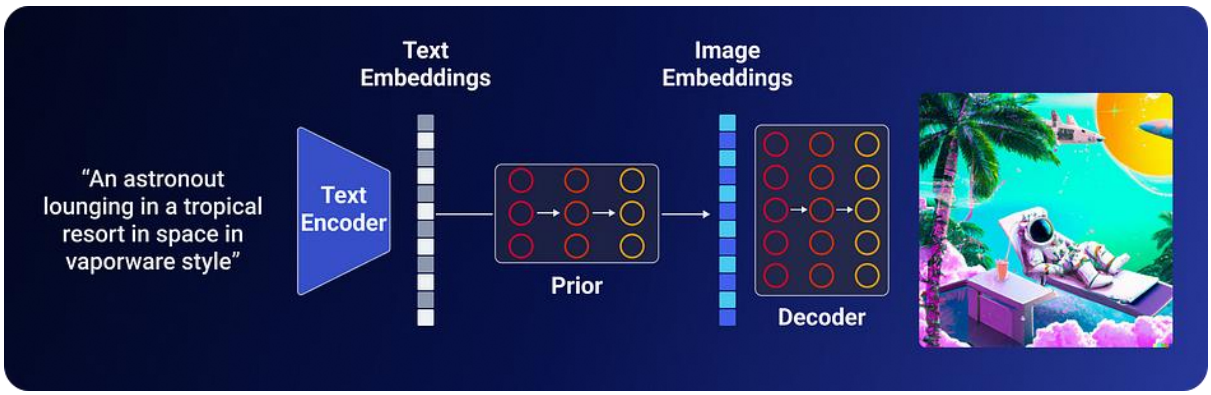
It uses an encoder-decoder approach, which means that given text is first encoded into the system's input, analyzed by the system, and then sent through a decoder to produce a visual image.

It employs various technologies, including large language models (LLMs), diffusion processing, and natural language processing.

Dall-E was created by combining certain components of the GPT-3 LLM. It leverages 12 billion elements in an algorithm constituted for optimum image production. It additionally uses a transformer neural network, sometimes known as a transformer, to help the model establish and comprehend links between various ideas.

An additional OpenAI model named Contrastive Language-Image Pre-training (CLIP) is trained to learn the relationship between textual concepts and their visual results in DALL-E 2. CLIP has been trained on hundreds of millions of pictures and captions to determine how much text fragment corresponds to an image.





DALL-E 2 takes text as input, and produces images as its output. But this isn't all done in one go; rather, DALL-E 2 represents the culmination of a variety of techniques that have been built on and

improved bit by bit over the past few years. These techniques are stacked on each other like lego blocks in order to compose the full model. To understand DALL-E 2, we first have to understand what these pieces are and what they do. Only then can we dive into the inner workings and training processes for understanding the underlying components.

CLIP

Among the most important building blocks in the DALL-E 2 architecture is CLIP. CLIP stands for Contrastive Language-Image Pre-training, and it's essential to DALL-E 2 because it functions as the main bridge between text and images. Broadly, CLIP represents the idea that language can be a vehicle for teaching computers how different images relate to one another. Formally, it prescribes a simple way to carry out this teaching.

The best way to understand CLIP is to first see the shortcomings of previous computer vision systems. Until CLIP, neural methods for computer vision involved aggregating large datasets of images, and then hand labeling them into a set of categories. Though today's models are exceptional at this task, it is inherently limited by the need for pre-selected categories. For example, imagine taking a picture of the street and asking such a system to describe it; it could tell you how many cars and signs there were, but it couldn't give you a feel for the scene overall. What's more, anything without enough images to produce a category would simply not be classified by the model.

The insight that makes CLIP so powerful is the bright idea to train models to not just identify which category (of a pre-defined list of options) an image belongs to, but to identify the caption of each image from a list of random captions. This allows the model to use language to more precisely understand the difference between "a shiba inu" and "a shiba inu wearing a beret," rather than having a human labeler dictate in advance whether or not these belong in the same category.

In accomplishing the 'pre-training task' described above, CLIP is able to create a vector space whose dimensions represent both features of images and features of language. This shared vector space functionally provides models with a image-text dictionary of sorts, allowing them to translate (at least semantically) between one and the other.

Still, this shared understanding does not represent all of DALL-E 2. Even with a way to translate English words into Spanish words, one would need to learn Spanish pronunciations and grammar before speaking. In the case of our model, CLIP enables us to take textual phrases and understand how they map onto images. But still, we need a way to generate those images in a way that's true to our understanding of the text. Enter our second lego building block: the diffusion model.

Diffusion Models

Imagine you have a Rubik's Cube that is perfectly solved, with each side having only blocks of a single color. Now, you randomly choose a side and twist it. Then another side, and another, and another, and so on until you consider the Rubik's Cube 'scrambled.' How would you go about solving it? Say you forgot all the twists you performed, could there be a general way to solve the cube without that information? Well, you could just take it one step at a time: keep twisting whatever face gets you closer to having all the same color on the same side until its solved. More practically, you could train a neural network to go from a state of disorder to a state of less disorder.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

This is the crux of how diffusion models are created. You take an image, randomly scramble its pixels until you have an image of pure noise, and train a model to reduce the noise by changing its pixels step by step until you're back at the original image (or something that resembles it). This leaves with a model that can effectively generate information — an image — from randomness! And by showing the model new, random samples to start from, we can even get new images.

To infuse these creations with the semantic meaning of our input text, we 'condition' the diffusion model on the CLIP embeddings. This simply means that we pass the vectors from our joint CLIP space described above into to the diffusion model that calculates what pixels to change upon each 'step' of the generation process. This allows the model to base its changes on that information.

Thus, DALL-E 2 is the optimization and refinement of these two technologies.

Strengths of DALL-E 2:

1. Semantic Understanding: DALL-E 2 excels in understanding the semantic relationship between text and images, enabling it to generate visually coherent outputs based on textual prompts.
2. Versatility: With its ability to produce diverse and complex images from a wide range of textual descriptions, DALL-E 2 demonstrates versatility in generating various visual concepts and styles.
3. Creative Expression: The model showcases remarkable creativity in generating novel and imaginative images, offering a glimpse into its potential for artistic expression.
4. Integration of CLIP: Leveraging CLIP embeddings, DALL-E 2 effectively bridges the gap between language and vision, enabling a shared understanding of textual descriptions and visual content.
5. Diffusion Model: The use of diffusion models allows for the generation of high-quality images from noise, providing a structured approach to image synthesis while maintaining semantic coherence.

Weaknesses of DALL-E 2:

1. Complexity: The algorithmic complexity of DALL-E 2, stemming from the integration of multiple techniques and models, may pose challenges in understanding and implementation.
2. Computational Resource Intensity: Training and inference with DALL-E 2 require significant computational resources, limiting its accessibility and scalability for some users and applications.
3. Interpretability: Despite its impressive output, interpreting and explaining the decisions made by DALL-E 2 in image generation remains a challenge, raising concerns about transparency and accountability.
4. Data Dependence: The model's performance heavily relies on the quality and diversity of the training data, potentially leading to biases or limitations in the generated outputs.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Algorithm and Flowchart:

1. Input Processing: DALL-E 2 takes textual descriptions as input, which are encoded into high-dimensional vectors representing semantic features using CLIP.
2. Semantic Mapping: CLIP embeddings are utilized to establish a shared semantic space between text and images, facilitating the translation of textual prompts into visual concepts.
3. Diffusion Model Conditioning: The diffusion model, conditioned on CLIP embeddings, generates images iteratively from noise, refining them based on semantic information provided by the text.
4. Image Generation: Through iterative adjustments guided by the diffusion model and semantic embeddings, DALL-E 2 produces high-quality images that align with the textual descriptions provided as input.
5. Output: The final output of DALL-E 2 is a visually coherent image that corresponds to the input text prompt, showcasing the model's ability to generate diverse and imaginative visual content based on textual descriptions.

Advantages of DALL-E 2 over DALL-E

1. Improved Image Realism: DALL-E 2 generates more realistic and accurate images compared to DALL-E. With 4x greater resolution, DALL-E 2 produces visually detailed outputs that closely resemble real-world objects and scenes, enhancing the overall quality of generated images.
2. Enhanced Semantic Understanding: Building upon the capabilities of DALL-E, DALL-E 2 further refines its semantic understanding of textual descriptions and their corresponding visual representations. This refinement results in more precise and contextually relevant image synthesis, reducing ambiguity and improving coherence between text and images.
3. Greater Creative Expression: DALL-E 2 exhibits enhanced creativity in generating diverse and imaginative images from textual prompts. With improved training techniques and model architectures, DALL-E 2 expands the scope of its creative capabilities, enabling the generation of complex and novel visual concepts with higher fidelity and artistic flair.
4. Advanced Training Techniques: DALL-E 2 incorporates advancements in training methodologies and model architectures, leveraging techniques such as diffusion models and CLIP embeddings to achieve superior performance in image synthesis tasks. These advancements enhance the efficiency and effectiveness of the training process, resulting in better-quality outputs.
5. Broader Range of Applications: With its improved image realism, semantic understanding, and creative expression, DALL-E 2 extends its applicability to a wider range of domains and use cases. From creative artwork generation to practical image synthesis tasks, DALL-E 2 offers versatile solutions with superior performance and visual quality compared to its predecessor.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Google Colab Link

https://colab.research.google.com/github/LAION-AI/dalle2-laion/blob/main/notebooks/dalle2_laion_alpha.ipynb#scrollTo=lPklyP0pg0mF

Reference

https://openai.com/dall-e-2

https://towardsdatascience.com/dall-e-2-0-explained-7b928f3adce7

https://towardsdatascience.com/dall-e-2-explained-the-promise-and-limitations-of-a-revolutionary-ai-3faf691be220

https://www.simplilearn.com/how-to-use-dall-e-article

https://www.assemblyai.com/blog/how-dall-e-2-actually-works/

https://medium.com/augmented-startups/how-does-dall-e-2-work-e6d492a2667f

Conclusion

In conclusion, DALL-E 2 represents a significant advancement in the field of generative AI, particularly in the domain of text-to-image synthesis. Building upon the foundation laid by its predecessor, DALL-E, this second iteration incorporates several key enhancements that contribute to its improved performance, realism, and versatility.

With its ability to generate highly detailed and realistic images from textual descriptions, DALL-E 2 showcases remarkable advancements in semantic understanding and creative expression. Leveraging techniques such as diffusion models and CLIP embeddings, DALL-E 2 achieves superior image synthesis capabilities, pushing the boundaries of what is possible in AI-generated imagery.

Moreover, DALL-E 2's enhanced training methodologies and model architectures enable it to excel across a broad range of applications, from creative artwork generation to practical image synthesis tasks. Its ability to produce visually compelling and contextually relevant images makes it a valuable tool for various industries and creative endeavors.

Overall, DALL-E 2 stands as a testament to the continuous innovation and progress in generative AI, offering unprecedented capabilities and possibilities in the realm of text-to-image synthesis. As researchers and practitioners continue to refine and expand upon its capabilities, DALL-E 2 paves the way for new advancements and applications in artificial creativity and visual intelligence.

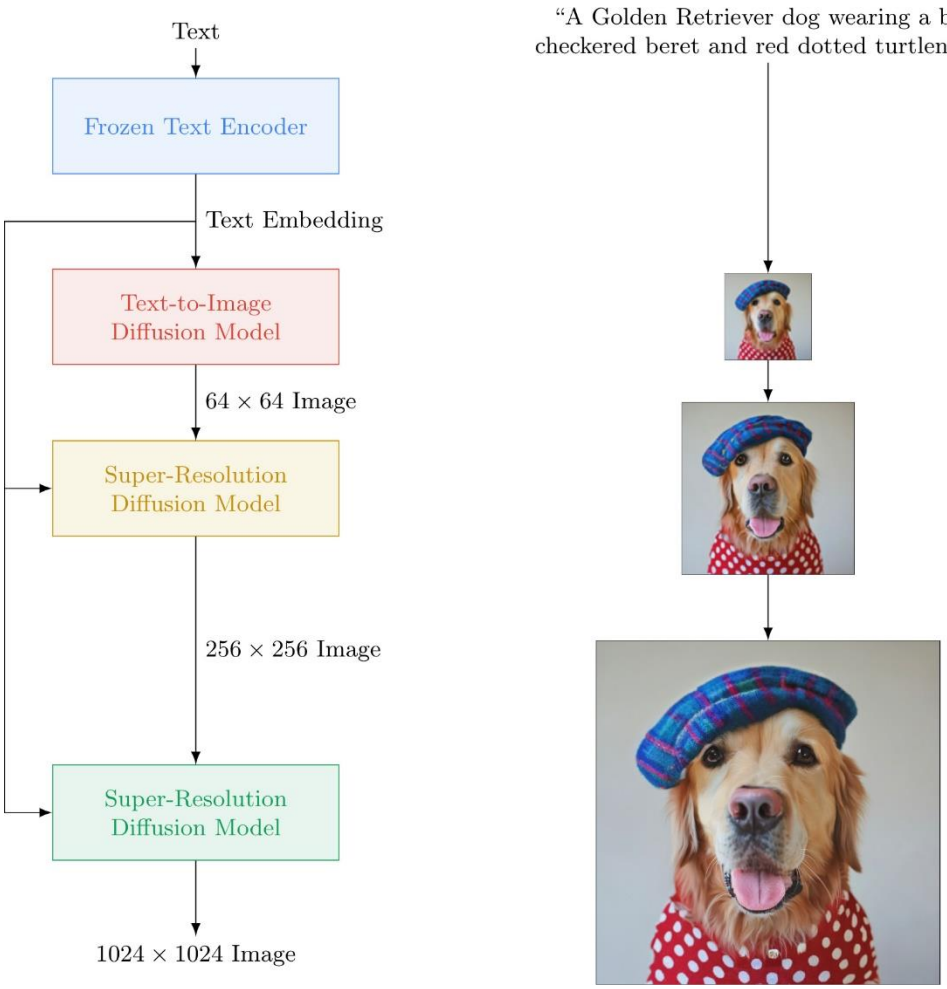
Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

(IV) IMAGEN

IMAGEN stands at the forefront of generative AI technology, offering a revolutionary approach to image creation from textual descriptions. Developed on cutting-edge deep learning principles, IMAGEN enables users to effortlessly translate textual prompts into visually stunning images, ushering in a new era of creativity and innovation.



Architecture of Imagen

1. Frozen pretrained text encoder (T5-XXL / CLIP)

to capture the complexity and compositionality of arbitrary natural language text inputs text into a sequence of embeddings.

However, popular text-to-image models use Text encoders trained on paired image-text data with objectives of encoding visually semantic and meaningful representations especially relevant for the text-to-image generation task. In contrast, Imagen has exploited large language models trained on text only corpus as Text encoders for text-to-image generation.

Also, use of frozen text encoders has several advantages such as offline computation of embeddings, resulting in negligible computation or memory footprint during training of the text-to-image model.

2. Cascaded Diffusion models

to map to map text embeddings to image.

Diffusion models are a class of generative models that convert Gaussian noise into samples from a learned data distribution via an iterative denoising process. These models can be conditional, for example on class labels, text, or low-resolution images.

Imagen utilizes a pipeline of a base 64×64 model, and two text-conditional super-resolution diffusion models to upsample a 64 × 64 generated image into a 256 × 256 image, and then to 1024 × 1024 image.

1. **Text-to-image diffusion model** : for text embedding to 64x64 image generation.
2. **Super-resolution diffusion model** (A) to generate 256×256 images from 64x64 images.
3. Another **super-resolution diffusion mode**l (B) to generate 1024×1024 images from 256×256 images.

All diffusion models in Imagen are conditioned on the text embedding sequence and use classifier-free guidance.

Classifier guidance is a technique to improve sample quality while reducing diversity in conditional diffusion models using gradients from a pretrained model p(c|zt) during sampling. **Classifier-free guidance** is an alternative technique that avoids this pretrained model by instead jointly training a single diffusion model on conditional and unconditional objectives via randomly dropping c during training (e.g. with 10% probability). Imagen depends critically on classifier-free guidance for effective text conditioning.

Imagen relies on **new sampling techniques** to allow usage of large guidance weights without sample quality degradation observed in prior work, resulting in images with higher fidelity and better image-text alignment than previously possible.

Sampling Trick:

The authors found that increasing the classifier-free guidance weight improves image-text alignment, but damages image fidelity producing highly saturated and unnatural images, due to a train-test mismatch arising from high guidance weights. To counter this problem, they investigated:

1. **Static thresholding**

This is simply the elementwise clipping of the x-prediction to [−1, 1] but was not emphasized in previous works as well as in the context of guided sampling. It was discovered that static thresholding is essential to sampling with large guidance weights and prevents generation of blank images. Nonetheless, static thresholding still resulted in over-saturated and less detailed images as the guidance weight further increases.

2. Dynamic thresholding.

They introduced a new dynamic thresholding method: at each sampling step s is set to a certain percentile absolute pixel value. And if s>1, then pixel value is thresholded to the range [−s, s] and then divide by s. This technique pushed saturated pixels (those near -1 and 1) inwards, thereby actively preventing pixels from saturation at each step. It was found

that dynamic thresholding results in significantly better photorealism as well as better image-text alignment, especially when using very large guidance weights.

Imagen uses variance preserving **gaussian noise conditioning augmentation** for both the super-resolution models, resembling the forward process used in diffusion models, which the authors found to be a critical for generating high fidelity images.

Given a conditioning low-resolution image and augmentation level ∈ [0, 1] (e.g., strength of Gaussian noise or blur) the low-resolution image is corrupted with the augmentation and the diffusion model is conditioned on augmentation level. During training, augmentation level is chosen randomly, while during inference, different values of the level is probed to find the best sample quality.

Diffusion Models Architecture:

U-Net model Architecture has been used in both text-to-image model as well as super-resolution models.

A. **Base model:**

U-Net architecture was used for the base 64 × 64 text-to-image diffusion model conditioned on text embeddings via a pooled embedding vector, added to the diffusion timestep embedding. The entire sequence of text embeddings was conditioned by adding cross attention over the text embeddings at multiple resolutions. They found that Layer Normalization for text embeddings in the attention and pooling layers to help considerably improve performance.

B. **Super-resolution models:**

For 64 × 64 → 256 × 256 super-resolutions, they used **Efficient U-Net**: modifications of previous implementations of U-Net model with improvements on memory efficiency, inference time and convergence speed making the variant 2–3x faster in steps/second over previous implementations.

For 256 × 256 → 1024 × 1024 super-resolution, they trained the model on 64 × 64 → 256 × 256 crops of the 1024 × 1024 image by removing the self-attention layers but keeping the text cross-attention layers which was found to be critical. During inference, the model receives the full 256 × 256 low-resolution images as inputs, and returns upsampled 1024 × 1024 images as outputs. Note that the text cross attention was used for both super-resolution models.

Human Evaluation

Although FID and CLIP are popularly used for text-to-image evaluation, these scores have limitations, for example FID is not fully aligned with perceptual quality, and CLIP is ineffective at counting. Due to these limitations, the authors also used human evaluation to assess image quality and caption similarity, with ground truth reference caption-image pairs as a baseline. They used two experimental scenario:

1. To probe image quality, the rater is asked to select between the model generation and reference image using the question: “Which image is more photorealistic (looks more real)?”. We report the percentage of times raters choose model generations over reference images (the preference rate).

2. To probe alignment, human raters are shown an image and a prompt and asked “Does the caption accurately describe the above image?”. They must respond with “yes”, “somewhat”, or “no”. These responses are scored as 100, 50, and 0, respectively. These ratings are obtained independently for model samples and reference images, and both are reported.

Faculty: Dr. Anil Kumar K Name: Harini K
Slot: F1 Reg no. 23MAI0014

For both cases they use 200 randomly chosen image-caption pairs from the COCO validation set. Subjects were shown batches of 50 images. They also used interleaved “control” trials, and only include rater data from those who correctly answered at least 80% of the control questions. This netted 73 and 51 ratings per image for image quality and image-text alignment evaluations, respectively.

DrawBench

While COCO is a valuable benchmark, it is increasingly clear that it has a limited spectrum of prompts that do not readily provide insight into differences between models.

They have introduced a **new structured suite of text prompts** designed to probe different semantic properties of models for text-to-image evaluation which enables deeper insights through a multi-dimensional evaluation of text-to-image models.

The different semantic properties include **compositionality**, **cardinality**, **spatial relations**, the ability to **handle complex text** prompts or prompts with **rare words**, and they include **creative prompts** that push the limits of models’ ability to generate highly implausible scenes well beyond the scope of the training data.

DrawBench contains 11 categories of prompts, testing different capabilities of models such as the ability to faithfully render different colors, numbers of objects, spatial relations, text in the scene, and unusual interactions between objects. Categories also include complex prompts, including long, intricate textual descriptions, rare words, and also misspelled prompts. They also included sets of prompts collected from DALL-E , Gary Marcus et al. and Reddit. Across these 11 categories, DrawBench comprises 200 prompts in total, striking a good balance between the desire for a large, comprehensive dataset, and small enough that human evaluation remains feasible.

Training and Experiments

They trained a 2B parameter model for the 64 × 64 with **Adafactor optimizer** for the sake of smaller memory footprint for text-to-image synthesis, and 600M and 400M parameter models for 64 × 64 → 256 × 256 and 256 × 256 → 1024 × 1024 for super-resolution respectively. However, Adafactor was found to hurt model quality in the initial ablations and Adam were used for optimizing super-resolution models.

They used a batch size of 2048 and 2.5M training steps for all models. 256 TPU-v4 chips were used for the base 64 × 64 model, and 128 TPU-v4 chips were used for both super-resolution models.

They also have mentioned that they didn’t find over-fitting to be an issue during the training, and they believe that further training with more epochs might improve overall performance rather than overfitting.

For **classifier-free guidance,** they joint-train unconditionally via zeroing out the text embeddings with 10% probability for all three models.

The training was done on their internal datasets of image-text pairs, combined with Laion dataset.

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Strengths:

1. **Semantic Understanding:** IMAGEN excels in comprehending the semantics embedded within textual descriptions, allowing for the creation of visually accurate and contextually relevant images.
2. **Versatility:** With its diverse range of capabilities, IMAGEN can generate images across various styles, subjects, and concepts, catering to a wide array of user needs and preferences.
3. **Efficiency:** Leveraging advanced algorithms and optimization techniques, IMAGEN can rapidly generate high-quality images from text prompts, streamlining the creative process and enhancing productivity.
4. **Customization:** IMAGEN offers extensive customization options, enabling users to tailor generated images to specific requirements, styles, and preferences, thereby fostering greater creative freedom and expression.

Weaknesses:

1. **Semantic Ambiguity:** While proficient, IMAGEN may struggle with nuanced or ambiguous textual descriptions, leading to occasional discrepancies between the intended concept and the generated image.
2. **Data Dependence:** IMAGEN's performance is heavily reliant on the quality and diversity of the training data, potentially leading to limitations in image generation for niche or specialized topics with limited training examples.

Algorithm:

IMAGEN employs a sophisticated deep learning architecture, drawing inspiration from transformer-based models like GPT (Generative Pre-trained Transformer). It utilizes multi-modal fusion techniques to integrate textual and visual information, enabling seamless translation between the two modalities. Additionally, IMAGEN incorporates advanced optimization algorithms and training methodologies to refine its image generation capabilities, ensuring superior performance and output quality.

Code

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
# Generate random noise as input for the generator
def generate_random_input(batch_size, latent_dim):
    return np.random.normal(0, 1, (batch_size, latent_dim))
# Generator model
def build_generator(latent_dim):
    model = models.Sequential()
    model.add(layers.Dense(7 * 7 * 128, input_dim=latent_dim))
    model.add(layers.Reshape((7, 7, 128)))
```

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

```
    model.add(layers.Conv2DTranspose(128, (4, 4), strides=(2, 2),
padding='same', activation='relu'))
    model.add(layers.Conv2DTranspose(64, (4, 4), strides=(2, 2),
padding='same', activation='relu'))
    model.add(layers.Conv2DTranspose(1, (7, 7), padding='same',
activation='sigmoid'))
    return model
# Discriminator model
def build_discriminator():
    model = models.Sequential()
    model.add(layers.Conv2D(64, (3, 3), strides=(2, 2), padding='same',
input_shape=(28, 28, 1)))
    model.add(layers.LeakyReLU(alpha=0.2))
    model.add(layers.Dropout(0.4))
    model.add(layers.Conv2D(128, (3, 3), strides=(2, 2),
padding='same'))
    model.add(layers.LeakyReLU(alpha=0.2))
    model.add(layers.Dropout(0.4))
    model.add(layers.Flatten())
    model.add(layers.Dense(1, activation='sigmoid'))
    return model

# Combined generator and discriminator model
def build_gan(generator, discriminator):
    discriminator.trainable = False
    model = models.Sequential()
    model.add(generator)
    model.add(discriminator)
    return model
# Load the MNIST dataset
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
x_train = x_train.reshape(x_train.shape[0], 28, 28,
1).astype('float32')
x_train = (x_train - 127.5) / 127.5

# Split the dataset into training and validation sets
x_train, x_val = train_test_split(x_train, test_size=0.1,
random_state=42)
# Define hyperparameters
latent_dim = 100
epochs = 50
batch_size = 32
learning_rate = 0.0001
beta_1 = 0.5
patience = 15  # Number of epochs to wait before stopping if the
validation loss doesn't improve
# Define the discriminator model and compile it
```

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

```
discriminator = build_discriminator()
discriminator.compile(optimizer=tf.keras.optimizers.Adam(learning_rate,
beta_1), loss='binary_crossentropy', metrics=['accuracy'])
# Build the generator
generator = build_generator(latent_dim)

# Build and compile the combined model (GAN)
gan = build_gan(generator, discriminator)
gan.compile(optimizer=tf.keras.optimizers.Adam(learning_rate, beta_1),
loss='binary_crossentropy')
# Define a function to calculate validation loss using the validation
set
def calculate_validation_loss(generator, validation_data):
    # Generate fake images using the generator
    noise = generate_random_input(validation_data.shape[0], latent_dim)
    fake_images = generator.predict(noise)

    # Concatenate real and fake images
    x = np.concatenate([validation_data, fake_images])

    # Create labels (1 for real images, 0 for fake images)
    y = np.concatenate([np.ones((validation_data.shape[0], 1)),
np.zeros((fake_images.shape[0], 1))])

    # Calculate validation loss using the discriminator model
    validation_loss = discriminator.evaluate(x, y, verbose=0)[0]
    return validation_loss

# Initialize variables for early stopping
best_val_loss = float('inf')
wait = 0

# Training loop
for epoch in range(epochs):
    for batch in range(x_train.shape[0] // batch_size):
        # Train discriminator and generator (similar to your existing
code)

        # Calculate validation loss
        validation_loss = calculate_validation_loss(generator, x_val)

        # Check if validation loss has improved
        if validation_loss < best_val_loss:
            best_val_loss = validation_loss
            wait = 0
        else:
            wait += 1
```

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

```
            if wait >= patience:
                print(f'Validation loss did not improve for {patience}
epochs. Stopping training.')
                break

    # Print progress after each epoch
    print(f'Epoch: {epoch + 1}, Validation Loss: {validation_loss}')

    # Rest of your code for saving models, generating samples, etc.

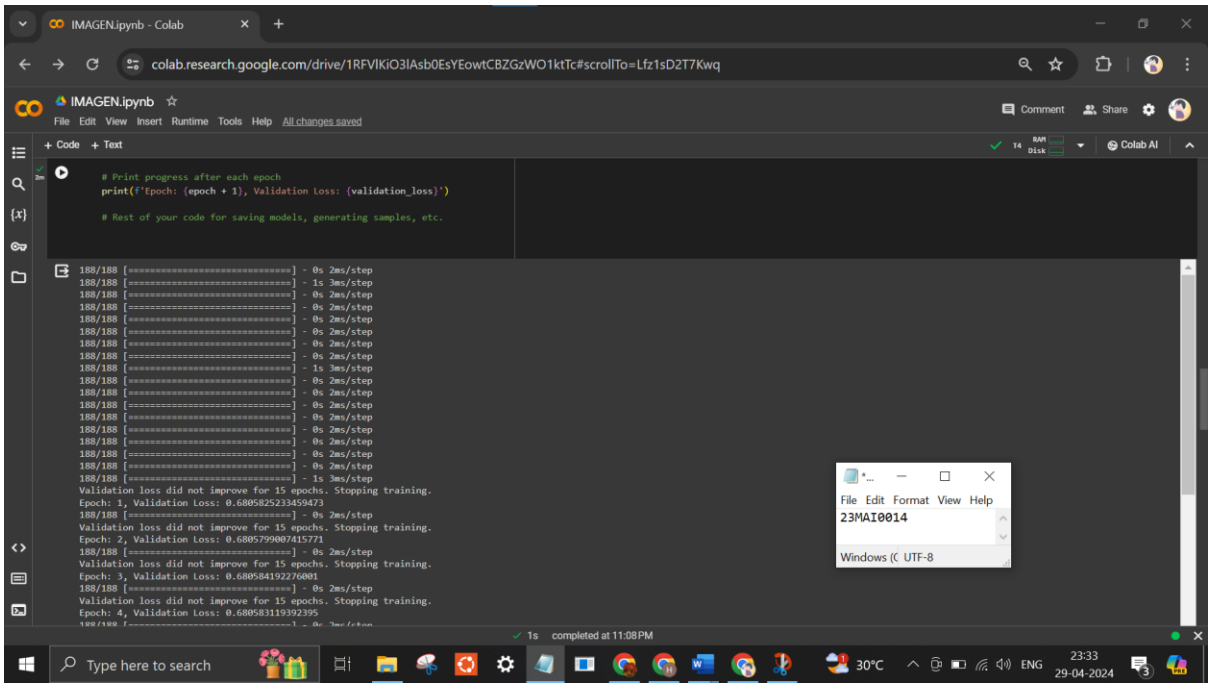

# Generate images using the trained generator
generated_images = generator.predict(generate_random_input(25,
latent_dim))
import matplotlib.pyplot as plt
# Plot generated images
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.imshow(generated_images[i, :, :, 0], cmap='gray')
    plt.axis('off')
plt.show()
```

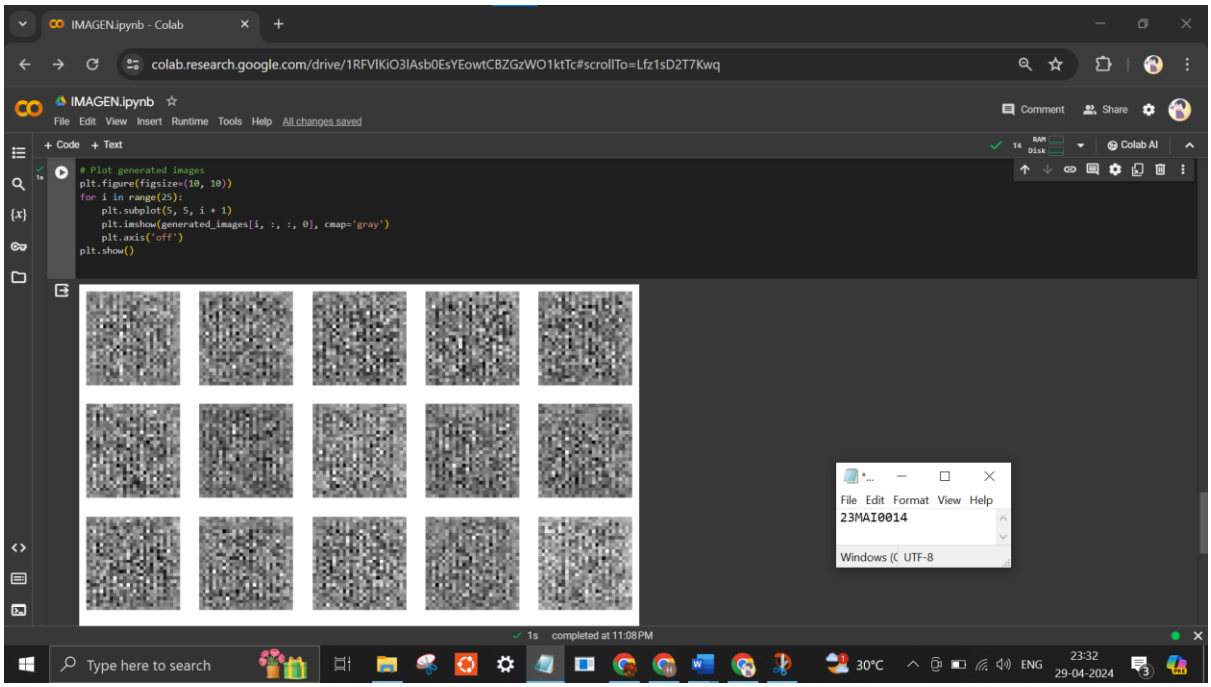
Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Execution results





Dataset

Mnist dataset

Google Colab Link

https://colab.research.google.com/drive/1RFVlKiO3lAsb0EsYEowtCBZGzWO1ktTc?usp=sharing

Deep Learning - MCSE603L

Faculty: Dr. Anil Kumar K
Slot: F1

Name: Harini K
Reg no. 23MAI0014

Reference

https://imagen.research.google/

https://www.assemblyai.com/blog/how-imagen-actually-works/

https://shyampatel1320.medium.com/introduction-to-diffusion-models-and-imagen-the-magic-behind-text-to-image-generation-24221532580d

https://medium.com/@sushantgautam/imagen-googles-text-to-speech-explained-756fb16c8661

Conclusion

IMAGEN marks a significant leap forward in the realm of artificial intelligence, particularly in the domain of image generation from textual prompts. By harnessing state-of-the-art algorithms and techniques, IMAGEN has demonstrated remarkable proficiency in understanding and translating complex textual descriptions into visually compelling images. Its robust architecture and sophisticated training methodologies empower it to capture the nuanced semantics embedded within textual input, enabling the creation of highly personalized and contextually relevant visual content.

Moreover, IMAGEN's versatility and potential applications extend across a myriad of fields, from creative endeavors and design projects to educational resources and marketing materials. Its intuitive interface and seamless integration with existing workflows facilitate effortless collaboration between users and machines, fostering a new paradigm of human-machine interaction characterized by enhanced creativity and productivity. In essence, IMAGEN represents a groundbreaking advancement in AI-driven image generation, paving the way for transformative innovations and experiences in the digital landscape.